

 吗喽笔记多 关注

 51   208  2  分享 ...

专栏目录

【TCP通信】原理详解与编程实现（一）



GitCode 开源社区 文章已被社区收录

加入社区



计算机相关 专栏收录该内容

6 篇文章

订阅专栏

文章目录

- 1.TCP简介
- 2.TCP报文
- 3.TCP建立与断开连接
 - 3.1 TCP的三次握手
 - 3.2 TCP的四次挥手
- 4.TCP数据传输示意
- 5.TCP如何实现可靠传输
 - 5.1 停止等待方式
 - 5.2 流水线传输方式
 - 5.3 可靠传输实例：
- 6.TCP流量控制
- 7.TCP拥塞控制
- 8.TCP相关面试题
- 参考

本文主要是对韩立刚计算机网络视频课程TCP通信部分的总结和整理，并参考了其它相关博客。

本文主要是理论知识。

下一篇文章将进行编码实现 [【TCP通信】原理详解与编程实现（二）](#)

1.TCP简介

TCP协议，**传输控制协议**（英语：Transmission Control Protocol，缩写为：TCP）是一种面向连接的、可靠的、基于字节流的通信协议

TCP把连接作为最基本的抽象单元，每条TCP连接有两个端点，**TCP连接的端点即套接字**。

套接字 socket = （IP地址+端口号）

TCP连接={socket1, socket2}={(IP1:port1), (IP2,port2)}

TCP提供全双工通信。

2.TCP报文



吗喽笔记多

关注

51



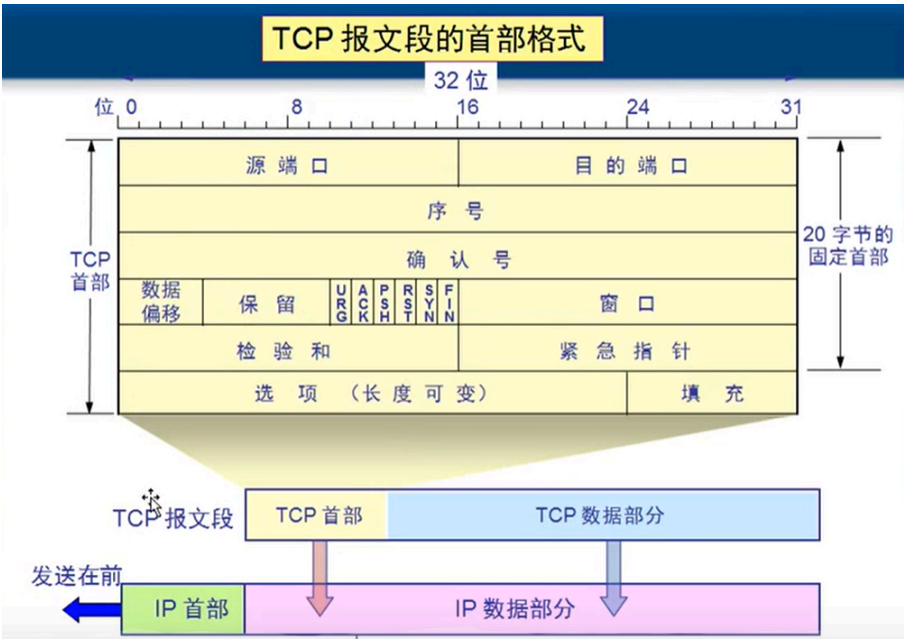
208

2

分享



专栏目录



首部中的重要概念

序号：Seq序号，占32位。用于说明当前数据第一个字节在所有数据(整个文件)中的位置

确认号：Ack序号，占32位。用于告诉发送者接下来需要发送的数据序号。

数据偏移：用于说明首部长度。(比如1111说明首部长为15*4字节)

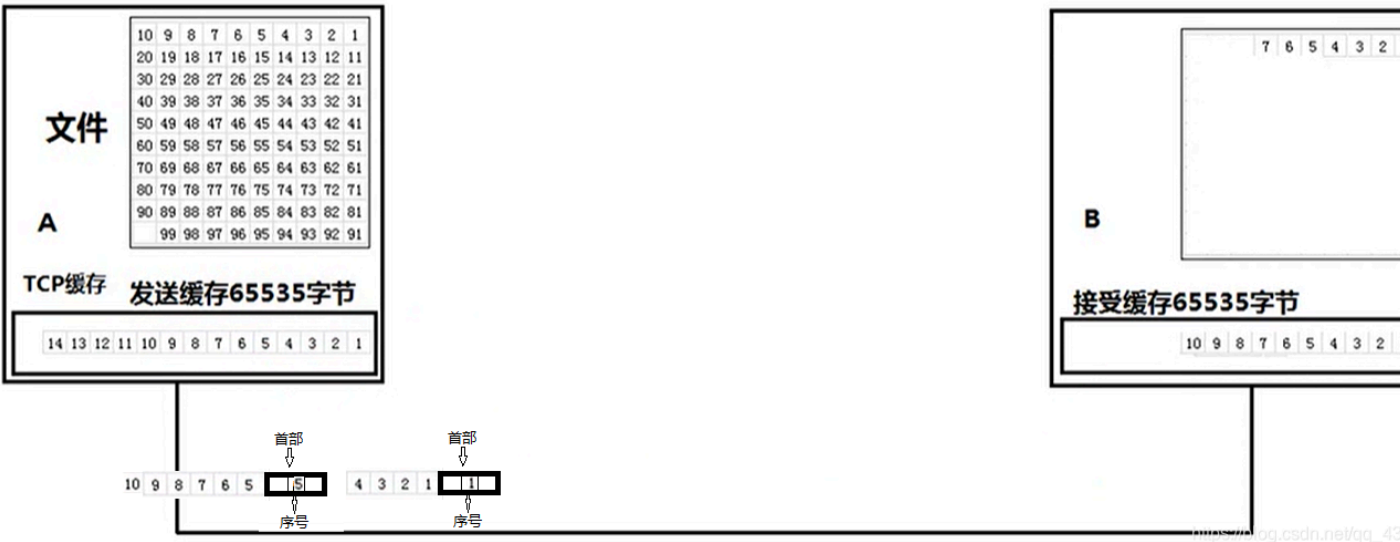
标志位：tcp标志位有6种：

cppAI写代码复制

```
1 SYN(synchronous发起一个新连接)
2 ACK(acknowledgement 确认)
3 PSH(push传送)
4 FIN(finish结束)
5 RST(reset重置)
6 URG(urgent紧急)
```

不要将确认序号Ack与标志位中的ACK搞混

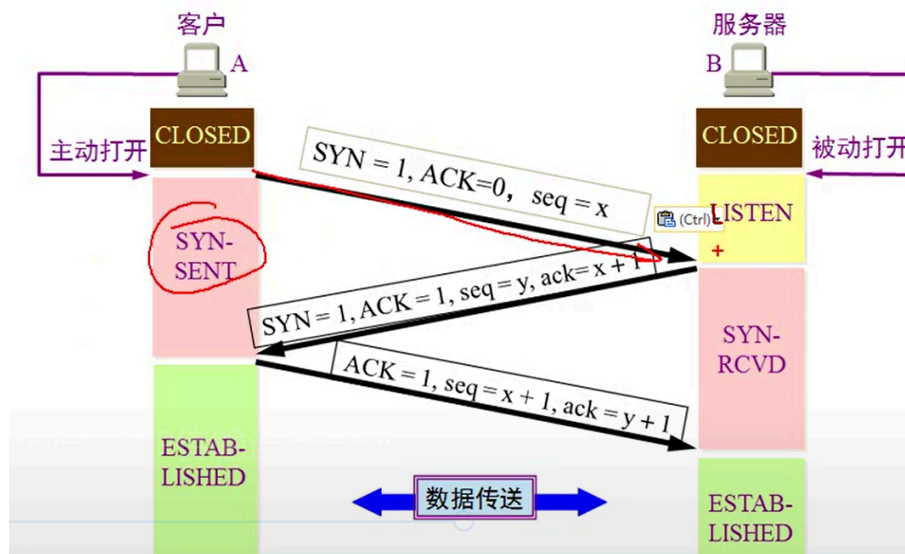
窗口：表示发送/接收缓存窗口大小。如下图。



3.TCP建立与断开连接

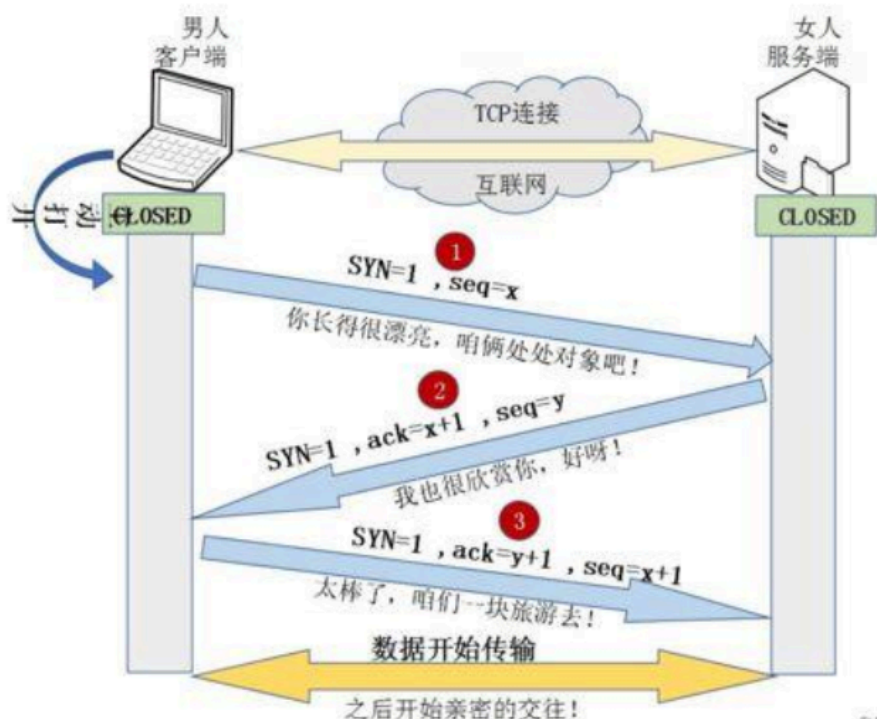
3.1 TCP的三次握手

三次握手其实就是建立连接的过程。其过程如图：



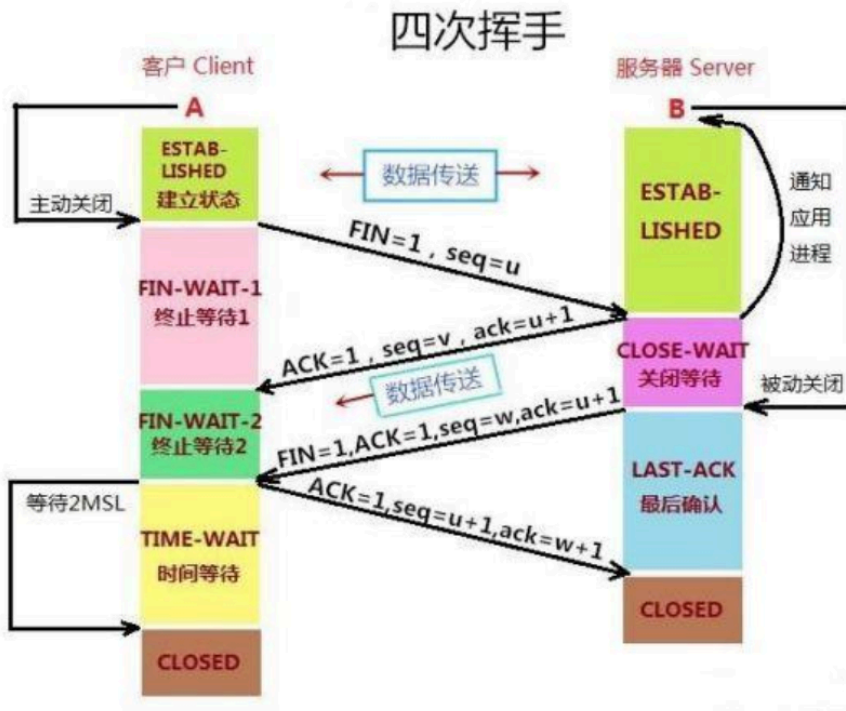
- 1) 第一次握手：Client将标志位**SYN**(建立新连接)置为1，随机产生一个值 $seq=x$ ，并将该数据包发送给Server，Client进入SYN_SENT状态，等待Server响应。
- 2) 第二次握手：Server收到数据包后由标志位 $SYN=1$ 知道Client请求建立连接，Server将标志位**SYN**和**ACK**(确认)都置为1， $ack=x+1$ ，随机产生一个值 $seq=y$ ，并将该数据包发送给Client以确认连接请求，Server进入SYN_RCVD状态。
- 3) 第三次握手：Client收到确认后，检查 ack 是否为 $x+1$ ，**ACK**是否为1，如果正确则将标志位**ACK**置为1， $ack=y+1$ ，并将该数据包发送给Server，Server检查 ack 是否为 $y+1$ ，**ACK**是否为1，如果正确则连接建立成功，Client和Server进入ESTABLISHED状态，完成三次握手，随后Client与Server之间可以开始传输数据了。

三次握手的通俗理解：



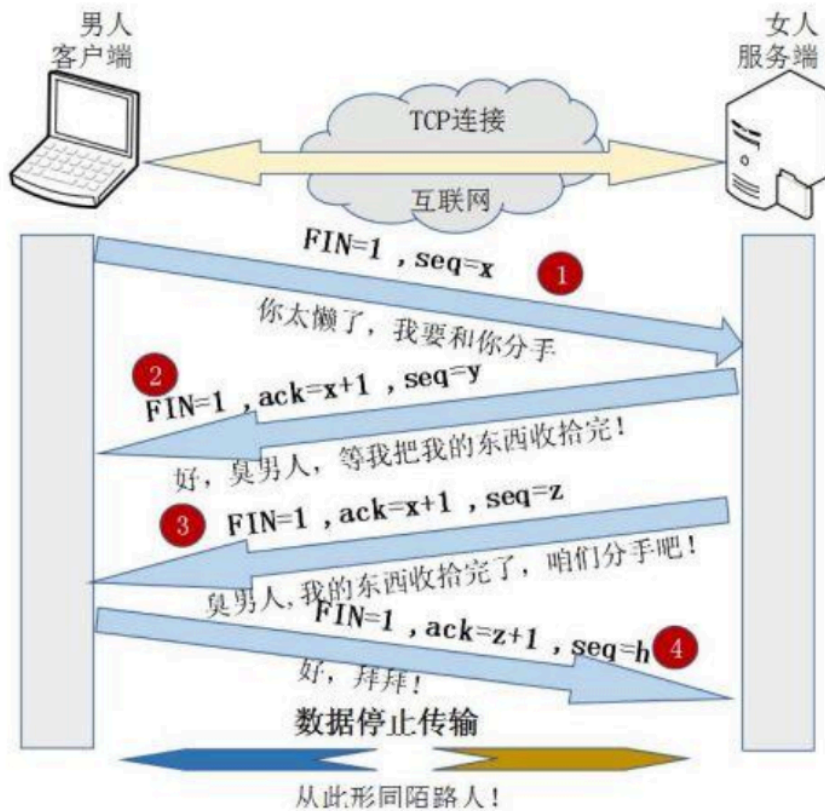
3.2 TCP的四次挥手

四次挥手其实就是断开连接的过程。过程如图：



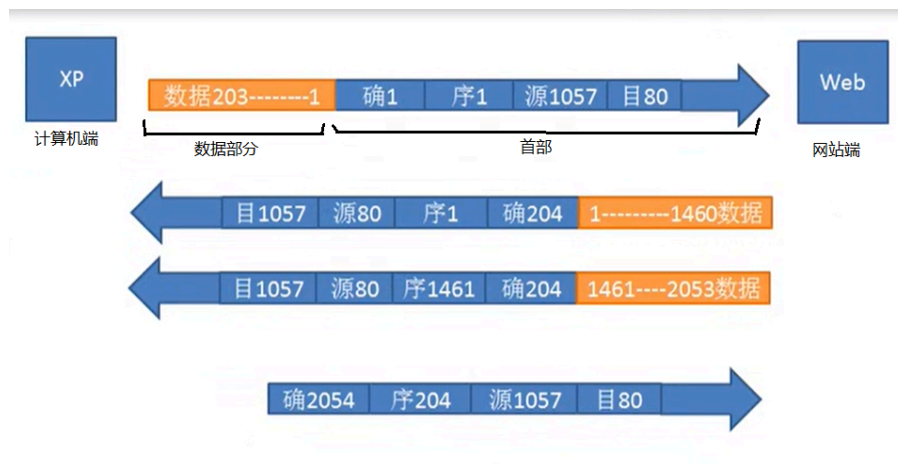
- 1) 第一次挥手：客户端向服务器发起请求释放连接的TCP报文，置FIN为1。客户端进入终止等待-1阶段。
- 2) 第二次挥手：服务器端接收到从客户端发出的TCP报文之后，确认了客户端想要释放连接，服务器端进入CLOSE-WAIT阶段，并向客户端发送一段TCP文。客户端收到后进入终止等待-2阶段。
- 3) 第三次挥手：服务器做好了释放服务器端到客户端方向上的连接准备，再次向客户端发出一段TCP报文。。此时服务器进入最后确认阶段。
- 4) 第四次挥手：客户端收到从服务器端发出的TCP报文，确认了服务器端已做好释放连接的准备，于是进入时间等待阶段，并向服务器端发送一段报文。
意：第四次挥手后客户端不会立即进入closed阶段，而是等待2MSL再关闭。

四次挥手的通俗理解：



4.TCP数据传输示意

以下图为例(计算机访问一个网站)说明传输过程:



- 1) 计算机访问网站, 源端口为1057, 目标端口为80, 序号为1, 确认号为1, 数据长203字节。
- 2) 网站返回数据, 确认号为204, 发送1-1460数据。
- 3) 网站接着发1461-2053数据, 确认号依然为204。(表明计算机发的203个字节已经接收到)
- 4) 计算机确认收到2053字节的数据, 但自身发送的数据为空。

5.TCP如何实现可靠传输

5.1 停止等待方式

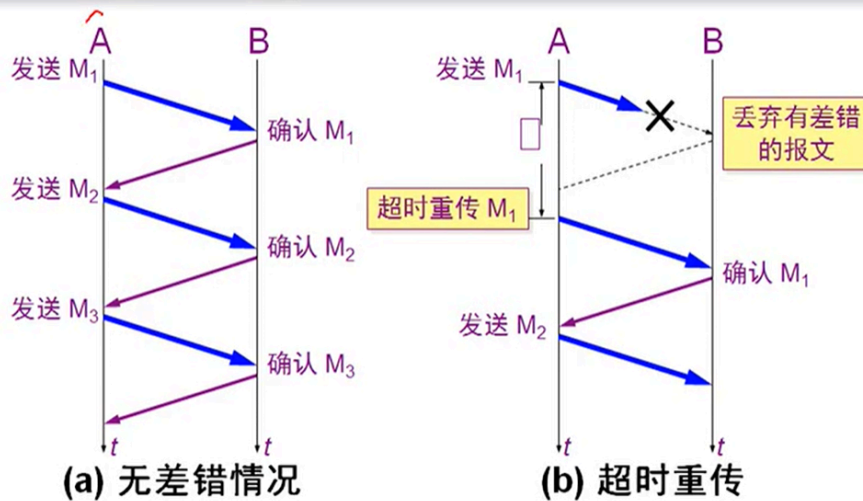
需要停止等待确认, 若设定时间内未收到确认则:

📖 吗喽笔记多 关注

👍 51 🗨 208 💬 2 🔄 分享 ...

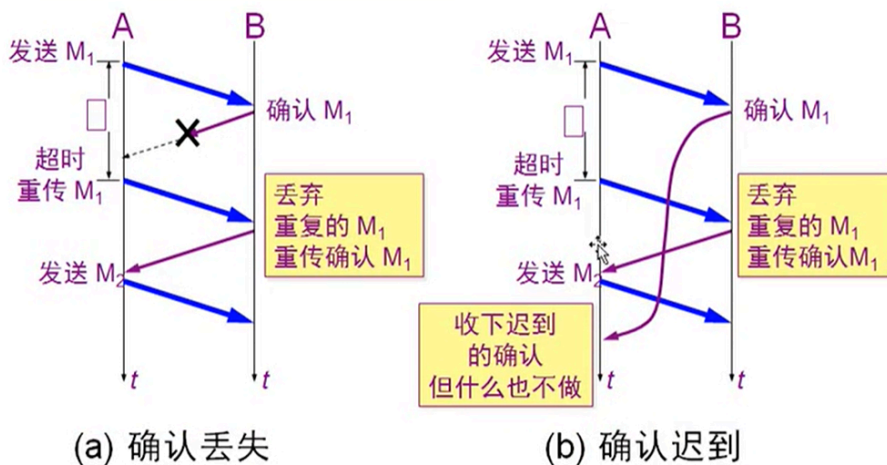
📁 专栏目录

可靠传输的工作原理-----停止等待协议



确认丢失与确认迟到:

确认丢失和确认迟到



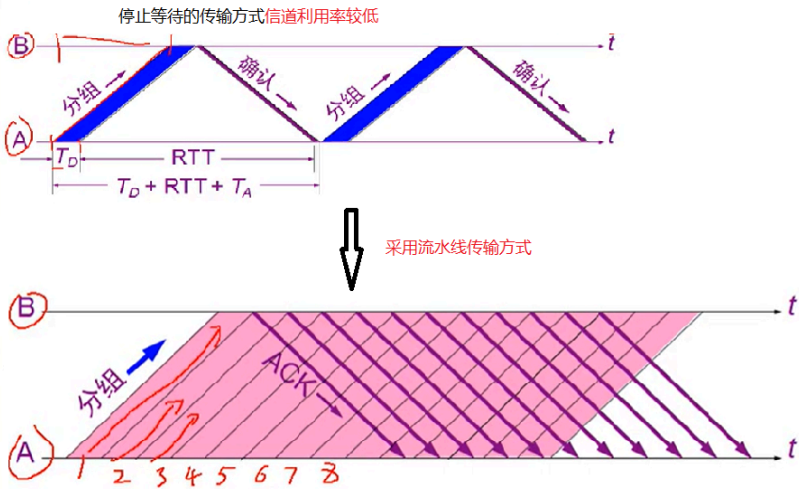
上述的确认和重传机制，可以称为自动重传机制：**ARQ**(Automatic repeat reQuest)

但是简单的停止等待方式信道利用率较低，为此采用流水线传输方式。

5.2 流水线传输方式

流水线传输：

发送方可连续发送多个分组，不必每发完一个分组就停顿下来等待对方的确认。由于信道上一直有数据不间断的传送，这种传输方式可获得很高的信道利

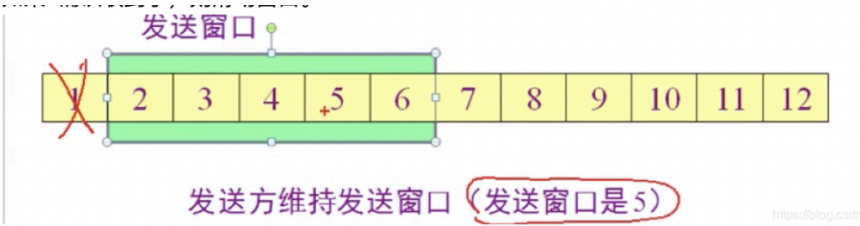


连续ARQ协议 / 滑动窗口：

上述发送端是维持滑动窗口实现流水线传输的，如下图：



如果1确认收到了，则滑动窗口。



累计确认：

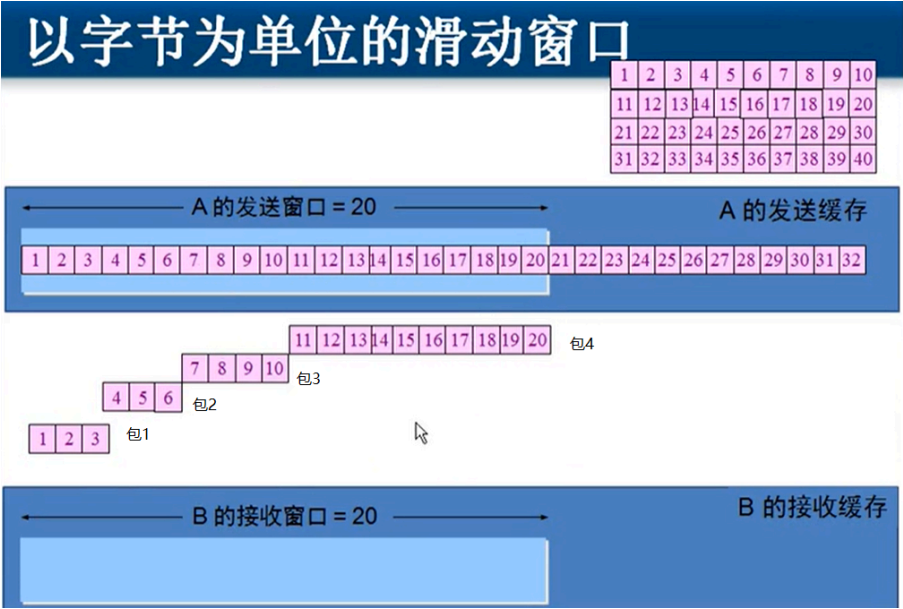
接收方一般采用累计确认的方式表明自己已经连续接收到哪个地方了。

5.3 可靠传输实例：

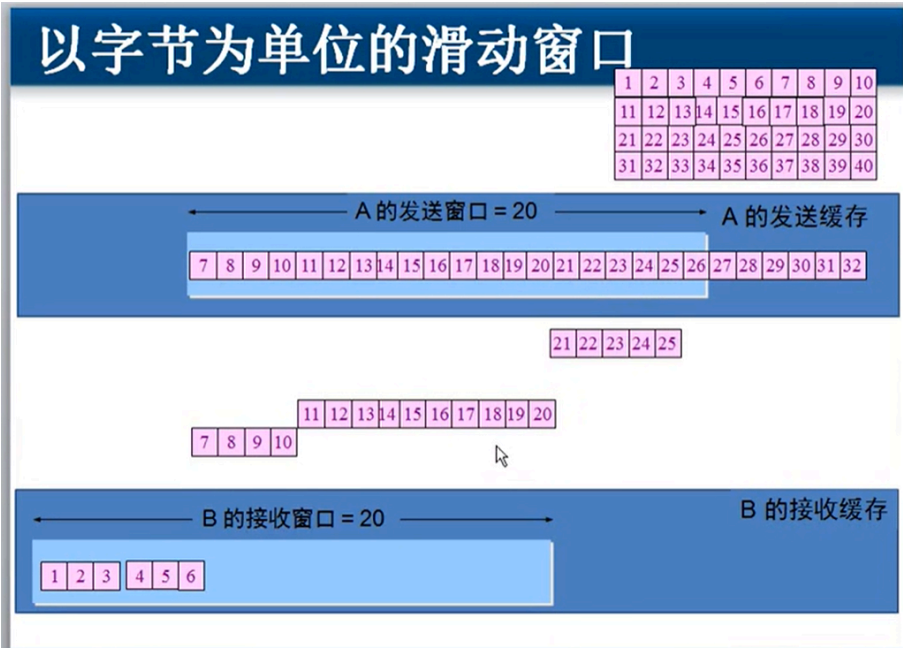
前面已经简单说明了tcp可靠传输的大致原理。这里具体举例说明。

假设，A计算机要发送一个文件(1-40字节)到B计算机。

1) 首先计算机A将缓存中的数据依次发送出去（多个数据包），在B确认之前将不会删除这些数据。



2)计算机B确认收到前两个数据包，并发送一个数据包(确认号为7)给A，于是A知道前6个字节已经发送成功，缓存窗口可以移动，舍弃前6个字节。

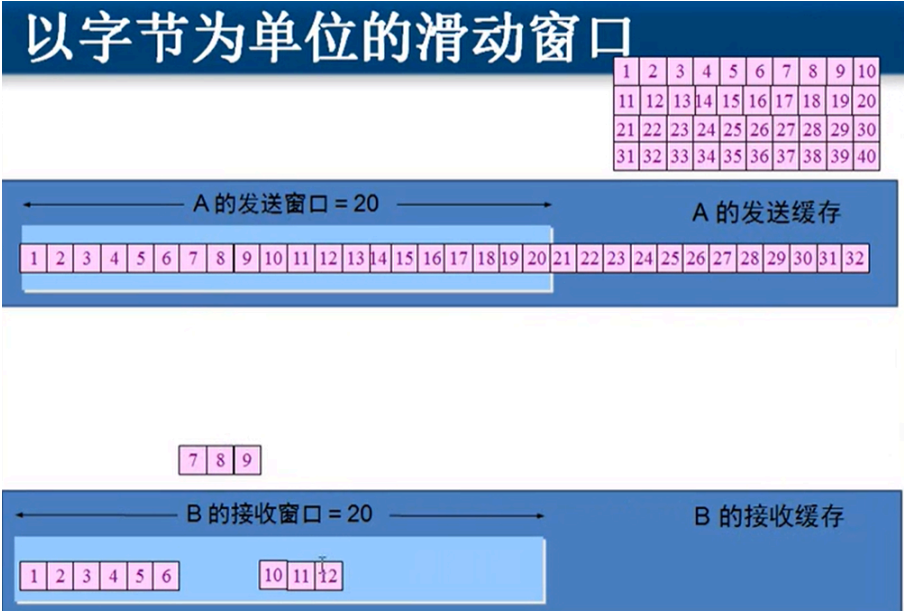


3)A继续发送窗口中的数据，B的接收窗口右移，B又接收到新数据，再向A确认新数据收到，A继续发送新数据。



4) 丢包的情况

假设A发的三个数据包中，只有第二个丢了，而一、三数据包没有丢，那么B只会确认第一个数据包，于是A超时重传第二个数据包，B重新接收到第二个包后，会直接确认第三个数据包，这样A就不用重传第三个数据包了。



6.TCP流量控制

TCP流量控制

概念：

接收端处理数据的速度是有限的，如果发送方的速度太快，就会把缓冲区u打满。这个时候如果继续发送数据，就会导致丢包等一系列连锁反应。所以TCF根据接收端能力来决定发送端的发送速度。这个机制叫做流量控制。

窗口大小：

（接收端向发送端主机通知自己可以接受数据的大小，这个大小限制就叫做窗口大小）

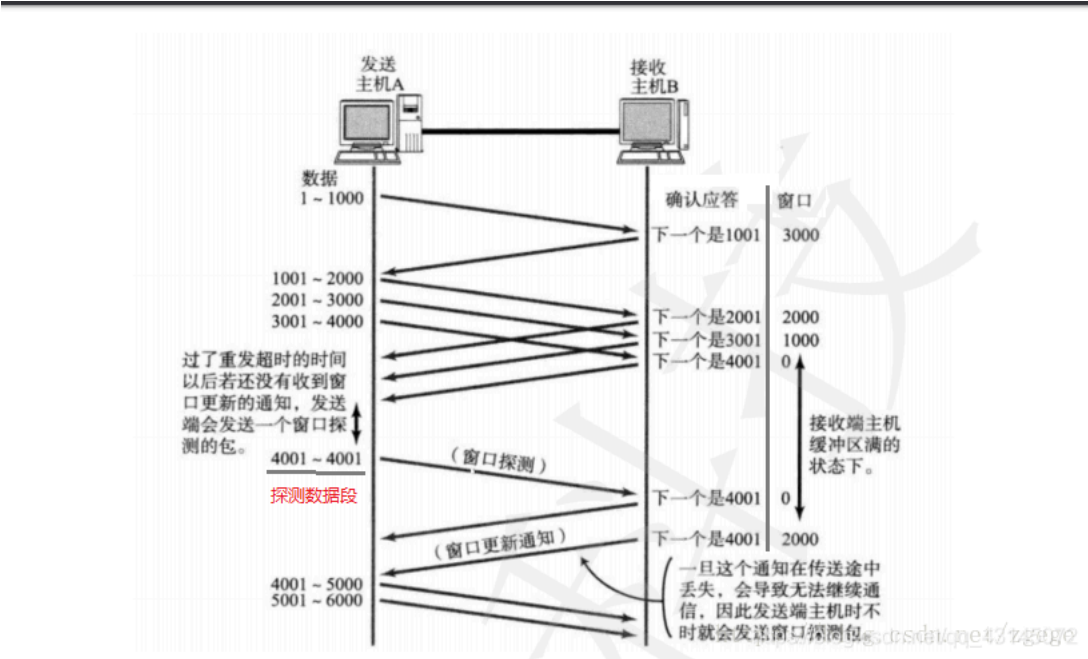
机理：

接收端将自己可以接收的缓冲区大小放入 TCP 首部中的“窗口大小”字段，通过ACK端通知发送端；窗口大小字段越大，说明网络的吞吐量越高；

接收端一旦发现自己的缓冲区快满了，就会将窗口

如果接收端缓冲区满了，就会将窗口置为0；这时发送方不再发送数据，但是需要定期发送一个窗口探测数据段，使接收端把最新的窗口大小告诉发送端，以便发送数据。

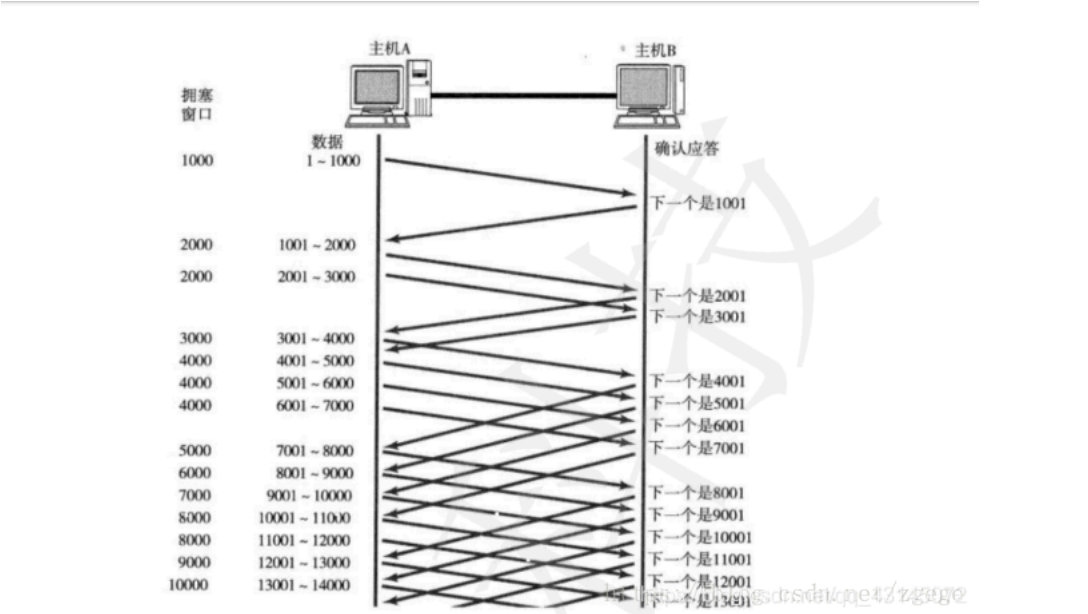
其过程示意如图：



7.TCP拥塞控制

在不清楚当前网络状态下，贸然发送大量的数据，可能引起计算机网络的拥堵。

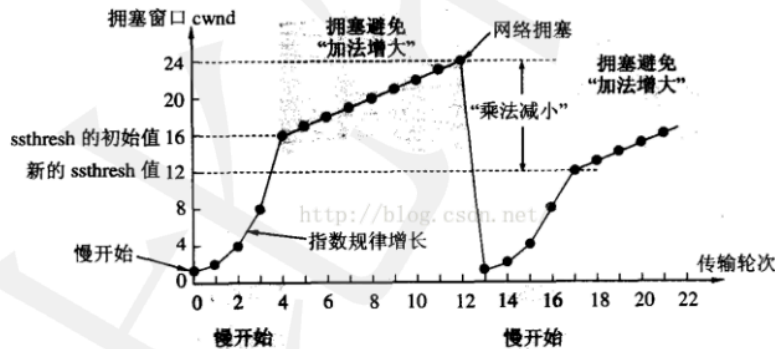
TCP引入慢启动机制，先发少量的数据，探探路，摸清当前的网络拥堵状态，再决定按照多大的速度传输。如下图：



每次发送数据包的时候，将拥塞窗口和接收端主机反馈的窗口大小做比较，取较小的值作为实际发送的窗口；像上面这样的拥塞窗口增长速度，是指数级别的。动”只是指初使时慢，但是增长速度非常快。为了不增长的那么快，因此不能使拥塞窗口单纯的加倍。此处引入一个叫做慢启动的阈值。

当拥塞窗口超过这个阈值的时候，不再按照指数方式增长，而是按照线性方式增长。

乘法减少：出现一次超时（即出现一次网络拥塞），就把慢开始门限值ssthresh设置为当前的拥塞窗口值乘以0.5。



8.TCP相关面试题

【问题1】为什么连接的时候是三次握手，关闭的时候却是四次握手？

答：因为当Server端收到Client端的SYN连接请求报文后，可以直接发送SYN+ACK报文。其中ACK报文是用来应答的，SYN报文是用来同步的。但是关闭连接时，当Server端收到FIN报文时，很可能并不会立即关闭SOCKET，所以只能先回复一个ACK报文，告诉Client端，“你发的FIN报文我收到了。有等到我Server端所有的报文都发送完了，我才能发送FIN报文，因此不能一起发送。故需要四步握手。

【问题2】为什么TIME_WAIT状态需要经过2MSL(最大报文段生存时间)才能返回到CLOSE状态？

答：虽然按道理，四个报文都发送完毕，我们可以直接进入CLOSE状态了，但是我们必须假象网络是不可靠的，有可能最后一个ACK丢失。所以TIME_WAIT状态就是用来重发可能丢失的ACK报文。在Client发送出最后的ACK回复，但该ACK可能丢失。Server如果没有收到ACK，将不断重复发送FIN片段。所以C能立即关闭，它必须确认Server接收到了该ACK。Client会在发送出ACK之后进入到TIME_WAIT状态。Client会设置一个计时器，等待2MSL的时间。如果期间内再次收到FIN，那么Client会重发ACK并再次等待2MSL。所谓的2MSL是两倍的MSL(Maximum Segment Lifetime)。MSL指一个片段在网络中最大的存间，2MSL就是一个发送和一个回复所需的最大时间。如果直到2MSL，Client都没有再次收到FIN，那么Client推断ACK已经被成功接收，则结束TCP连接。

【问题3】为什么不能用两次握手进行连接？

答：3次握手完成两个重要的功能，既要双方做好发送数据的准备工作(双方都知道彼此已准备好)，也要允许双方就初始序列号进行协商，这个序列号在握中被发送和确认。

现在把三次握手改成仅需要两次握手，死锁是可能发生的。作为例子，考虑计算机S和C之间的通信，假定C给S发送一个连接请求分组，S收到了这个分组并发送了确认应答分组。按照两次握手的协定，S认为连接已经成功地建立了，可以开始发送数据分组。可是，C在S的应答分组在传输中被丢失的情况下，将S是否已准备好，不知道S建立什么样的序列号，C甚至怀疑S是否收到自己的连接请求分组。在这种情况下，C认为连接还未建立成功，将忽略S发来的任何分组，只等待连接确认应答分组。而S在发出的分组超时后，重复发送同样的分组。这样就形成了死锁。

【问题4】如果已经建立了连接，但是客户端突然出现故障了怎么办？

TCP还有一个保活计时器，显然，客户端如果出现故障，服务器不能一直等下去，白白浪费资源。服务器每收到一次客户端的请求后都会重新复位这个计时器，时间通常是设置为2小时，若两小时还没有收到客户端的任何数据，服务器就会发送一个探测报文段，以后每隔75秒钟发送一次。若一连发送10个探测仍然没反应，服务器就认为客户端出了故障，接着就关闭连接。

参考

《计算机网络》视频课程韩立刚

详解 TCP 连接的“三次握手”与“四次挥手”

关于我们 招贤纳士 商务合作 寻求报道 400-660-0108 kefu@csdn.net 在线客服 工作时间 8:30-22:00

公安备案号11010502030143 京ICP备19004658号 京网文〔2020〕1039-165号 经营性网站备案信息 北京互联网违法和不良信息举报中心

家长监护 网络110报警服务 中国互联网举报中心 Chrome商店下载 账号管理规范 版权与免责声明 版权申诉 出版物许可证 营业执照

©1999-2025北京创新乐知网络技术有限公司